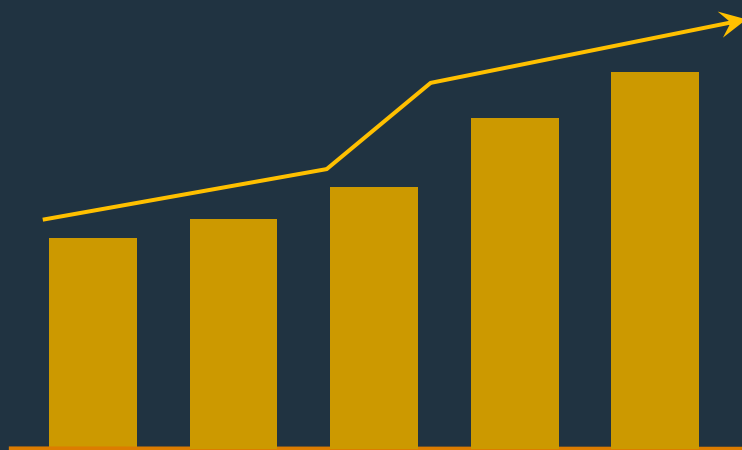




贪心

题目选讲

北京大学 王欣灏

堆的应用1：合并果子

有 n 堆果子，多多决定把所有的果子合成一堆。

每一次合并，多多可以把两堆果子合并到一起，消耗的体力等于两堆果子的重量之和。所有的果子经过 $n-1$ 次合并之后，就只剩下一堆了。合并果子时总共消耗的体力等于每次合并所耗体力之和。求这个最小的体力耗费值。

huffman树模型：

哈夫曼树一般是二叉树，建树的方法就是每次选择两个权值最小的点，删除这2个点，加入一个权值是这两个点之和的新点进去。并且使这被删除的2个点的父亲成为那个新点。

具体而言，将 n 堆果子看作 n 个数字加入到一个小根堆，每次合并相当于连续两次取出堆顶元素，将取出的两个数求和，再次加入到堆中。

堆的应用1：合并果子之 K叉哈夫曼树

有n堆果子，多多决定把所有的果子合成一堆。

每一次合并，多多可以把K堆果子合并到一起，消耗的体力等于K堆果子的重量之和。所有的果子经过若干次合并之后，就只剩下一堆了。合并果子时总共消耗的体力等于每次合并所耗体力之和。求这个最小的体力耗费值。

K叉huffman树模型：

K叉哈夫曼树是棵K叉树，建树的方法就是每次选择K个权值最小的点，删除这K个点，加入一个权值是这K个点之和的新点进去。并且使这被删除的K个点的父亲成为那个新点。

具体而言，将n堆果子看作n个数字加入到一个小根堆，每次合并相当于连续K次取出堆顶元素，将取出的K个数求和，再次加入到堆中。

然而每次选择k个权值最小的点的时候容易让最后一次合并的时候的点不足k个。假设最初有n个点，最后有1个点，每次合并删除k个点又放进1个点。那么易得： $(n-1)$ 是 $(k-1)$ 的倍数。如果 $(n-1) \% (k-1) \neq 0$ ，那么就要再放入 $(k-1 - (n-1) \% (k-1))$ 个虚拟点，并且它们的权值为0，它们也参与求最小k个点。

堆的应用2：钓鱼 洛谷1717

何老板打算钓鱼 h 小时，他家附近有 n 个池塘，这些池塘分布在一条直线上，何老板将它们按离家的距离编号，依次为 $L_1, L_2, \dots, L_n$ ，何老板家门口就是第1个池塘，所以他到第1个池塘是不用花时间的。

何老板可以任选多个池塘垂钓，并且在每个池塘他都可以呆上任意长的时间，但呆的时间必须为**5分钟的倍数**（5分钟为一个单位时间），他也可以在任意一个池塘结束今天的钓鱼活动。

已知从池塘 L_i 到池塘 L_{i+1} 要花去 t_i 个单位时间，每个池塘的上鱼率预先也是已知的，池塘 L_i 在第一个单位时间内能钓到的鱼为 F_i ，并且每过一个单位时间在单位时间内能钓到的鱼将**减少一个常数 d_i** ，现在请你编一个程序计算何老板最多能钓到多少鱼。

$$1 \leq h \leq 16 \quad 2 \leq n \leq 25 \quad 0 \leq d_i \leq 100 \quad 0 \leq F_i \leq 100 \quad 0 \leq d_i \leq 100$$

堆的应用2：钓鱼

问题分析：怎么处理来回奔走于各个池塘的情况？

问题答案：不会出现来回奔走的情况，**只会从左往右走**

因为在第*i*个池塘*x*分钟钓鱼后，离开一段时间后再次回到*i*号池塘钓鱼*y*分钟，一定没有直接在*i*号池塘钓*x+y*分钟优秀，因为来回要浪费时间，所以不会出现重复经过同一池塘的情况。

方法一：DP

阶段：从左往右依次讨论以每个池塘作为终点的情况

状态： **$f[i][j]$** 表示在*j*个单位时间内，在前*i*个池塘中最多能钓的鱼数

决策：在第*i*个池塘钓多长时间的鱼

方程： **$f[i][j] = \max\{ f[i-1][j], f[i-1][j-k-walk[i]] + GetFish[i][k] \}$**

$f[i-1][j]$ 表示不在第*i*个池塘钓鱼

$GetFish[i][k]$ 表示在第*i*个池塘钓*k*分钟能钓到的鱼数

$walk[i]$ 表示从*i-1*号池塘到达*i*号池塘行走花费的时间

$f[i-1][j-k-walk[i]]$ 总共*j*分钟，从*i-1*到*i*行走花费 $walk[i]$ 分钟，*i*号池塘钓鱼花费*k*分钟，剩余*j-k-walk[i]*分钟，这*j-k-walk[i]*是前*i-1*个池塘可以使用的钓鱼时间。

时间复杂度： $O(n*h*h)$

堆的应用2：钓鱼

方法二：贪心+堆

由于终点不固定，而且只能从左往右走，所以枚举终点，依次讨论每个点作为终点的情况：

假设以 i 号池塘作为终点，那么先用总时间减去从1号池塘到 i 号池塘行走耗费的时间， $\text{Fish_Time} = h - \text{walk}[i]$ ，那么剩下的时间(Fish_Time)就是用来钓鱼的时间了。

现在就相当于现在可以在1到 i 号池塘间任意瞬间移动了，每一个单位时间选当前能够钓到鱼最多的池塘来钓就行了，选最大用大根堆实现。

具体操作，先把1到 i 号池塘单位时间能够钓鱼的数量存入大根堆：

- 1.每次从堆顶取出能钓到最大数量鱼的湖，并在那里钓一个单位时间的鱼，将钓到的鱼累加钓鱼的总数上；
- 2.把该湖下一个单位时间能钓鱼的数量更新(减去 d_i)后，重新存入堆中；
- 3.总钓鱼时间减去一个单位的时间，重复该过程，直到没有时间或没有鱼可钓了为止；

时间复杂度： $O(n * h * \log h)$

堆的应用2：钓鱼 参考代码

```
struct node { int f,d; }a[30],T;           //a[i].f表示i号池塘单位时间的钓鱼量，a[i].d表示钓鱼量的减少常数

bool operator<(node a,node b) { return a.f<b.f; } //按照单位时间的钓鱼量建立大根堆

priority_queue <node> q;

main()
{
    .....
    for (i=1;i<=n;i++)                    //依次讨论以每个池塘作为钓鱼的终点
    {
        Sum= 0;
        Time = h - walk[i];                //h为钓鱼的总时间，walk[i]表示从起点到i号池塘行走耗费的时间
        for ( j=1;j<=i;j++)q.push(a[j]);    //将前i个池塘按单位时间钓鱼数量存入大根堆
        while (Time>0)                      //讨论每个单位时间在哪一个池塘钓鱼
        {
            T = q.top();                    //取出堆顶元素，堆顶代表当前单位时间内能够钓鱼最多的那个池塘
            q.pop();
            Sum = Sum + T.f;                 //Max记录下钓鱼总量。
            T.f =T.f - T.d;
            if (T.f<0) T.f = 0;              //注意：有可能出现单位时间钓鱼数量小于0的情况，应强行置0
            q.push(T);
            Time--;
        }
        if (Sum>ans) { ans = Sum; }          //更新答案
    }
    .....
}
```

堆的应用3：黑盒序列 洛谷1801

有一个只能存放整数的序列叫“黑盒序列”，一开始序列为空。对于该序列，我们有ADD和GET两种操作：

ADD(x)：表示往序列中添加一个值为x的整数；

GET(y)：表示在第y次ADD操作后，取出序列中第k小的一个数，并将其输出。（k初值为1，每执行一次GET操作后，k的值会加1）

$0 \leq N, M \leq 30000$

样例说明：

操作顺序	k	黑盒序列	输出结果
1 ADD(3)		3	
GET	1	3	3
2 ADD(1)		1, 3	
GET	2	1, 3	3
3 ADD(-4)		-4, 1, 3	
4 ADD(2)		-4, 1, 2, 3	
5 ADD(8)		-4, 1, 2, 3, 8	
6 ADD(-1000)		-1000, -4, 1, 2, 3, 8	
GET	3	-1000, -4, 1, 2, 3, 8	1
GET	4	-1000, -4, 1, 2, 3, 8	2
9 ADD(2)		-1000, -4, 1, 2, 2, 3, 8	

堆的应用3：黑盒序列

问题分析：在一个常常更新的数列中，常常去查找第k小数

方法一：用堆暴力枚举

对于每一个GET操作，我们把当前数列的所有数存入一个小根堆，然后用k次取出堆顶元素，其中第k次取出的堆顶元素就是数列的第k小数。

时间复杂度 $O(n*m*\log m)$

方法二：1个大根堆 + 1个小根堆

假设当前数列有t个数，我们怎么快速找目前第k小的数呢？

原理很简单：

- 1.将通过ADD操作新加入数列的数字全部存入大根堆(两次GET操作之间加入的数字)；
- 2.删除大根堆中最大的t-k+1个数，也就是最后大根堆中只留下了k-1个数字(也就是目前数列的前k-1小个数)；
- 3.在第2步中，每从大根堆里面删除一个数，就将该数加入到小根堆，最后小根堆里面有了t-k+1个数；
- 4.第3步完成后，小根堆的堆顶元素一定是当前这t个数中第k小的。
因为前k-1小的数在大根堆里面，于是将小根堆堆顶的数字删除，并将该数字加入到大根堆，大根堆里面就有了k个数了，也就是前k小的数字。
- 5.当讨论下一个GET操作时，也就是求第k+1小的数字，上一次GET操作前数列中的所有数字已存入了两个堆中，我们回到步骤1继续执行；

堆的应用3：黑盒序列 参考代码

```
.....
priority_queue<int,vector<int>,less<int> >Qmax;    //申明了一个大根堆
priority_queue<int,vector<int>,greater<int> >Qmin; //申明了一个小根堆
.....
Get[0]=0;
for(k=1;k<=m;k++)
{
    for(i=Get[k-1]+1;i<=Get[k];i++)Qmax.push(Add[i]); //将两次GET间新加入的数字入大根堆
    while(Qmax.size()>k-1)                             //大根堆中只保留目前前k-1小的数字
    {
        Qmin.push(Qmax.top()); //多的数字从大根堆中删除，并加入到小根堆
        Qmax.pop();
    }
    Qmax.push(Qmin.top()); //小根堆的堆顶元素一定是当前第k小数
    Qmin.pop();
    printf("%d\n",Qmax.top());
}
.....
```

堆的应用4：分配防晒霜 洛谷1801

有C头奶牛去日光浴，出门前她们要抹防晒霜。第i头奶牛适合的最小和最大的防晒值SPF分别为 minSPF_i 和 maxSPF_i 。如果某头奶牛涂的防晒霜的SPF值过小，那么阳光仍然能把她的皮肤灼伤；如果防晒霜的SPF值过大，则会使日光浴与躺在屋里睡觉变得几乎没有差别。

奶牛们准备L瓶防晒霜。第i瓶防晒霜的SPF值为 SPF_i 。第i瓶防晒霜可供 cover_i 头奶牛使用。每头奶牛只能涂某一个瓶子里的防晒霜，而不能把若干个瓶里的混合着用。

最多有多少奶牛能在不被灼伤的前提下，享受到日光浴的效果？

$1 \leq C, L \leq 2500$

$1 \leq \text{minSPF}_i \leq 1,000; \text{minSPF}_i \leq \text{maxSPF}_i \leq 1,000$

堆的应用4：分配防晒霜

问题分析：第*i*瓶防晒霜可供哪些奶牛使用？

我们可以考虑贪心的思想：

1. 在还没有分配防晒霜的奶牛中，将所有防晒值下限不大于第*i*瓶防晒霜防晒值 ($\min\text{SPF} \leq \text{SPF}[i]$) 的奶牛都找出来；
2. 从这些奶牛中选择防晒值上限不小于第*i*瓶防晒霜 ($\max\text{SPF} \geq \text{SPF}[i]$) 且该值尽可能小的奶牛，将第*i*瓶防晒霜分配给它们使用。
3. 实现上述操作，我们容易想到将这些奶牛按防晒值上限用小根堆维护即可。

具体实现：贪心+小根堆

1. 将奶牛按防晒值下限($\min\text{SPF}$)由小到大排序，将防晒霜按防晒值(SPF)由小到大排序；
2. 依次讨论每瓶防晒霜, 设当前讨论第*i*瓶: 将还没有分配防晒霜的且防晒值下限不大于 $\text{SPF}[i]$ 的奶牛存入小根堆，该堆以奶牛防晒值上限($\max\text{SPF}$)为关键字；
3. 依次取出堆顶元素，若堆顶元素的值($\max\text{SPF}$)不小于 $\text{SPF}[i]$ ，表示对应的牛可用第*i*瓶防晒霜，涂该牛后将第*i*瓶防晒霜可用的次数减一；
4. 在第3步完中，若取出的堆顶元素的值($\max\text{SPF}$)小于 $\text{SPF}[i]$ ，表示该牛无法涂到防晒霜。因为防晒霜是按防晒值由小到大顺序讨论的，当讨论到第*i*瓶防晒霜时，当前第*i*瓶也无法涂，后面的防晒霜也无法涂到了，因为后面防晒霜的防晒值都不小于第*i*瓶。而前面的*i*-1瓶防晒霜已讨论完毕，前*i*-1瓶该牛都没有被涂到，
5. 回到第2步，继续讨论第*i*+1瓶防晒霜；
6. 时间复杂度 $O(C \log C)$ 每头奶牛只有一次进堆的机会

堆的应用4：分配防晒霜 参考代码

```
k=1;
for(i=1; i<=L;i++)
{
    while (k<=C&&cow[k].minSPF<=cream[i].SPF)    //k标记当前讨论的奶牛的编号
                                                    //依次讨论每瓶防晒霜
    {
        while (k<=C&&cow[k].minSPF<=cream[i].SPF)    //选出防晒值下限不大于当前防晒霜的奶牛
        {
            Q.push(cow[k].maxSPF);
            k++;
        }

        while (!Q.empty()&&cream[i].cnt>0)    ///!Q.empty()表示还有防晒下限不超过i号防晒霜的奶牛没有分配霜
        {
            tmp=Q.top();
            Q.pop();
            if (tmp>=cream[i].SPF)    // 奶牛上限不小于第i瓶，说明这头奶牛可以被涂上i号防晒霜
            {
                ans++;
                cream[i].cnt--;
            }
            // else 这头奶牛不能被涂上，因为cream是按SPF排过序的，没有比这瓶更小的SPF了
        }
    }
}
```

植物大战僵尸 POJ3042

植物大战僵尸的游戏里有一条水平道路，道路的一端是入口，另一端是房子。僵尸会从道路的入口一端向房子一端移动。这条道路刚好穿过 N 块连续的空地。初始时，僵尸通过每块空地的时间是 T 秒。玩家可以在这 N 个空地中种植植物以攻击经过的僵尸，每块空地中只能种植一种植物。

共有三种不同类型的植物，分别是**红草**、**蓝草**和**绿草**，作用分别是**攻击**、**减速**以及**下毒**。每种植物只能在僵尸通过它所在空地的这段时间内攻击到僵尸。

当僵尸经过一块**红草**所在的空地时，**每秒钟生命值会减少 R 点**；当僵尸从一块**蓝草**所在的空地走出之后，**通过每块空地的时间延长 B 秒**；当僵尸从一块**绿草**所在的空地走出之后，**每秒钟会因中毒减少 G 点生命值**。蓝草的减速效果和绿草的下毒效果是可以累加的。也就是说，僵尸通过 n 块蓝草所在的空地之后，它通过每块空地的时间会变成 $T + B * n$ 秒；僵尸通过 n 块绿草所在的空地之后，它每秒钟会因中毒失去 $G * n$ 点生命值。注：**减速和中毒效果会一直持续下去**

问：怎样在这 N 块空地里种植各种类型的植物，才能使通过的僵尸失去的生命值最大。输出这个最大值。

$1 \leq N \leq 2000$ 空间限制 **3m**

例1：植物大战僵尸 问题分析

首先，一个显然的贪心是**红草一定排在最后**。

所以，若使用 rr 个红草，只须确定其余 $N-rr$ 个排在前面的蓝、绿草如何摆放，可以使用动规来解决

设 $f[gg][bb]$ 表示前面 $gg+bb$ 个位置使用 gg 个绿草、 bb 个蓝草可造成的最大伤害。

求解 $f[gg][bb]$ 只须讨论最后一格放的是何种草,可以由 $f[gg-1][bb]$ 和 $f[gg][bb-1]$ 递推得到。方程如下：

$f[gg][bb] = \max \{$

$f[gg-1][bb] + (T + bb*B)*(gg-1)*G;$

$f[gg][bb-1] + (T + (bb-1)*B)*gg*G;$

$\}$

经过第 $(gg+bb)$ 号格子时，中毒减少的生命值

$f[gg-1][bb] + (T + bb*B)*(gg-1)*G;$

表示最后一个位置放绿草,通过最后一格的时间是 $T + bb*B$ ，中毒造成的伤害是每时间 $(gg-1)*G$

$f[gg][bb-1] + (T + (bb-1)*B)*gg*G;$

表示最后一个位置放蓝草,通过最后一格的时间是 $T + (bb-1)*B$ ，中毒造成的伤害是每时间 $gg*G$

对于每个 $f[gg][bb]$ ，加上最后的 $N-gg-bb$ 个红草带来的伤害，以及在红草的区域由于中毒带来的伤害（后者易忽视），找到最优值即可

$\text{maxharm} = \max\{ f[gg][bb] + (N-gg-bb)*(T + bb*B)*(R + gg*G) \}$

例1：植物大战僵尸 参考代码

```
for(bb=0;bb<=N;bb++)
    for(gg=0;gg<=N-bb;gg++)
    {
        if(bb>0)f[bb][gg]=max(f[bb][gg],f[bb-1][gg]+(T+B*(bb-1))*gg*G);
        if(gg>0)f[bb][gg]=max(f[bb][gg],f[bb][gg-1]+(T+B*bb)*(gg-1)*G);
        if(N-bb-gg>0)maxharm=max(maxharm,f[bb][gg]+(N-bb-gg)*(T+bb*B)*(R+gg*G));
    }
cout<<maxharm;
```

提前计算对未来的影响

但本题空间限制只有3m,我们需要用**滚动数组**来优化空间